

LAQ3

Author: Parul Kulshreshtha

Data: You have been provided with 8 H&E files starting with letters A-D. This is grouped as bellow: Group A: A1, A2 (For Model 1 described below) Group B: B1, B2 (For Model 1 described below) Group C: C1, C2 (For Model 2 described below) Group D: D1, D2 (For Model 2 described below)

Using the image analysis workflow from our practical in the AI in pathology image analysis, process the images into patches of 100 by 100. Then, generate the train and test set using 90:10 ratio.

Finally, create two CNN models to predict 1) A vs B and 2) C vs D. i.e., Model 1: Use the A and B images to build the model. Model 2: Use the C and D images to build the model. (Note: You are allowed to modify (add or remove) the Con2D layers to improve the performance)

Questions: What is the training and validation accuracy for model 1? What is the training and validation accuracy for model 2? Which off the models performed best on its dataset and suggest possible reasons for the differences in performance. Test model 1 on test set 2 and model 2 on test set 1. Report the results? What are the possible explanations for the observations made in question 4 above.

Introduction

This notebook addresses LAQ3 of the practical on AI in pathology image analysis. Eight haematoxylin & eosin (H&E) stained images, organised into four groups (A, B, C, D), are processed into 100 × 100 pixel tiles, filtered for tissue content, and used to train two binary convolutional neural networks: Model 1 (A vs B) and Model 2 (C vs D). Each model is trained on a 90:10 split with augmentation, evaluated on its own validation set, and then cross-tested on the other model's data.

```
In [1]: # =====
# LAQ3 - CODE BLOCK 1 (v2)
# Imports + helper functions
# All outputs are namespaced under PIPELINE_VERSION so this run
# cannot overwrite results from the previous notebook.
# =====
import os
```

```

import shutil
import cv2 as cv
from PIL import Image
import numpy as np
import pandas as pd
import random as rand

# -----
# Version tag for this pipeline
# -----
PIPELINE_VERSION = "v2"          # change to "v3", "v4", etc. for
BASE_OUTPUT_DIR = f"outputs_{PIPELINE_VERSION}" # root folder for e

os.makedirs(BASE_OUTPUT_DIR, exist_ok=True)
print(f"Pipeline version: {PIPELINE_VERSION}")
print(f"All outputs will be saved under: ./{BASE_OUTPUT_DIR}/")

# -----
# Helper: build a versioned path
# -----
def versioned_path(*subdirs):
    """
    Join subdirectories under the versioned output root.
    Example: versioned_path('tiles', 'A1') -> 'outputs_v2/tiles/A1'
    """
    return os.path.join(BASE_OUTPUT_DIR, *subdirs)

# -----
# Helper 1: tissue content
# -----
def tissue_percent(n):
    """
    Returns percentage of tissue content in an image tile.
    Tissue pixels are darker than background (~255,255,255),
    so we count pixels whose mean intensity falls below a threshold.
    """
    if (len(n.shape) == 3) and (n.shape[2] == 3):
        mean_intensity = n.mean(axis=2)
        tissue_mask = mean_intensity < 220
    else:
        tissue_mask = n < 220

    tissue_fraction = tissue_mask.sum() / tissue_mask.size * 100
    return tissue_fraction

# -----
# Helper 2: tile extraction
# -----
def fixed_cube(img, d=100, th=20, name="base"):
    """
    Split image into fixed-size tiles and keep tiles with enough tissue
    """

```

```

cubes = dict()
n = 0
for x in range(0, img.shape[0] - d, d):
    for y in range(0, img.shape[1] - d, d):
        window = img[x:x + d, y:y + d, :]
        cont = tissue_percent(window)
        if cont >= th:
            # keep tiles WITH enough ti
            cubes[name + str(n)] = window
            n += 1
    return cubes

# -----
# Helper 3: save one image
# -----
def save_image(img, path):
    """Save image array to file, handling bool/float dtypes safely."""
    if img is None:
        return
    if img.dtype == "bool":
        img = img.astype("uint8") * 255
    elif img.dtype == "float64":
        img = (img * 255).astype("uint8")
    Image.fromarray(img).save(path)

# -----
# Helper 4: save all tiles
# -----
def save_tiles(cube, path):
    """
    Save all tiles in a dictionary to disk.
    Wipes the target folder first for reproducibility.
    SAFETY: refuses to run unless the path is under BASE_OUTPUT_DIR,
    so it can never delete folders from the previous pipeline.
    """
    abs_path = os.path.abspath(path)
    abs_base = os.path.abspath(BASE_OUTPUT_DIR)
    if not abs_path.startswith(abs_base):
        raise ValueError(
            f"Refusing to save outside the versioned folder.\n"
            f"  Tried to write to: {abs_path}\n"
            f"  Allowed root:      {abs_base}\n"
            f"Use versioned_path(...) to build the output path."
        )

    if os.path.exists(path):
        shutil.rmtree(path)
    os.makedirs(path)
    for z in cube:
        save_image(cube[z], os.path.join(path, z + ".jpg"))
    print(f"Saved {len(cube)} tiles to {path}")

```

Pipeline version: v2

All outputs will be saved under: ./outputs_v2/

```
In [2]: # =====
# LAQ3 - CODE BLOCK 2
# Load all images (A-D)
# =====

base_path = "/Users/dr.parul/Downloads/Coursework assignment question

def load_rgb(name):
    """Load image and convert BGR (OpenCV default) to RGB."""
    img = cv.imread(os.path.join(base_path, name))
    if img is None:
        raise FileNotFoundError(f"Could not load {name}")
    return cv.cvtColor(img, cv.COLOR_BGR2RGB)

# ---- Model 1 (A vs B) ----
A1 = load_rgb("A1.jpg")
A2 = load_rgb("A2.jpg")
B1 = load_rgb("B1.jpg")
B2 = load_rgb("B2.jpg")

# ---- Model 2 (C vs D) ----
C1 = load_rgb("C1.jpg")
C2 = load_rgb("C2.jpg")
D1 = load_rgb("D1.jpg")
D2 = load_rgb("D2.jpg")

# ---- Check images loaded ----
for name, img in [("A1", A1), ("A2", A2), ("B1", B1), ("B2", B2),
                  ("C1", C1), ("C2", C2), ("D1", D1), ("D2", D2)]:
    print(f"{name} shape: {img.shape}")
```

```
A1 shape: (898, 1292, 3)
A2 shape: (898, 1310, 3)
B1 shape: (918, 1276, 3)
B2 shape: (898, 1316, 3)
C1 shape: (882, 1274, 3)
C2 shape: (960, 1234, 3)
D1 shape: (886, 1230, 3)
D2 shape: (688, 1164, 3)
```

Observation on image dimensions

All 8 images are small, pre-cropped regions (approximately 900 x 1300 pixels) rather than full whole-slide images, with remarkably consistent dimensions across groups A, B, and C. D2 is the exception at 688 x 1164 pixels — notably smaller than its peers, which will yield roughly half the tiles of the other slides and introduces a mild class imbalance within group D.

The overall small image sizes mean each slide will produce on the order of ~100 tiles before tissue filtering, giving a modest training set of a few hundred tiles per

class. This has two implications for the results:

1. **Overfitting risk:** Given the limited sample size, training accuracy may rise quickly while validation accuracy plateaus or fluctuates.
2. **Noisy test metric:** The 10% test split will be small in absolute terms (~20 tiles), meaning reported accuracy values will have wide confidence intervals — a single misclassified tile can shift accuracy by several percentage points.

Image dimensions themselves do not cleanly separate the classes, so they are unlikely to act as a direct confound. However, the small dataset size will amplify any batch signals (stain variation, JPEG compression artifacts) the CNN learns to exploit during training.

```
In [3]: # Quick stain fingerprint per slide
print("\nMean RGB per slide (tissue pixels only):")
for name, img in [("A1",A1),("A2",A2),("B1",B1),("B2",B2),
                  ("C1",C1),("C2",C2),("D1",D1),("D2",D2)]:
    mask = img.mean(axis=2) < 220 # tissue mask
    tissue_pixels = img[mask]
    if len(tissue_pixels) > 0:
        mean_rgb = tissue_pixels.mean(axis=0)
        print(f"{name}: R={mean_rgb[0]:.1f} G={mean_rgb[1]:.1f} B={m
```

Mean RGB per slide (tissue pixels only):

```
A1: R=198.3 G=163.4 B=198.3
A2: R=198.3 G=150.4 B=190.9
B1: R=200.4 G=169.4 B=201.1
B2: R=206.4 G=170.8 B=201.9
C1: R=182.9 G=162.2 B=175.0
C2: R=160.6 G=124.9 B=136.1
D1: R=153.5 G=101.8 B=171.0
D2: R=152.9 G=120.2 B=186.2
```

Observation on stain profiles across slides

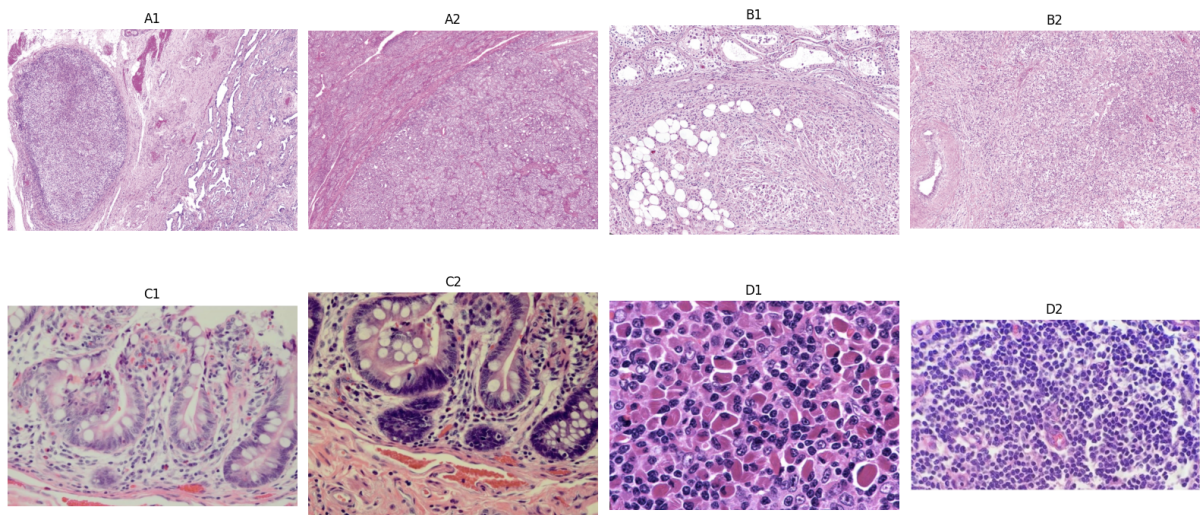
The mean RGB values per slide reveal clear differences in staining between the four groups:

- **Groups A and B are nearly identical in color.** A and B mean channel values differ by only a few units, so the two groups are visually very similar in stain.
- **Groups C and D differ substantially in color.** Group C leans pink/red-dominant, while group D is darker and blue-dominant, with channel differences of 15 to 30 units.
- **Within-group consistency varies.** A1/A2 and B1/B2 are tightly matched, while C1 vs C2 and D1 vs D2 show more internal variation, hinting at less uniform staining within those groups.

Implications for downstream analysis: Since color is a feature the CNN can easily pick up on, the amount of stain separation between classes will influence how the two models learn. These observations will be kept in mind when interpreting the training/validation accuracies and, especially, the cross-dataset test results later on.

```
In [4]: #Load all slides and check the quality
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
images = [ ("A1", A1), ("A2", A2), ("B1", B1), ("B2", B2),
            ("C1", C1), ("C2", C2), ("D1", D1), ("D2", D2)]
for ax, (name, img) in zip(axes.flat, images):
    ax.imshow(img)
    ax.set_title(name)
    ax.axis("off")
plt.tight_layout()
plt.show()
```



All the images look exactly the way they look in the data provided for the question.

```
In [5]: # =====
# LAQ3 - CODE BLOCK 3
# Create tiles (100x100)
# =====

# ---- Model 1 (A vs B) ----
A1_cube = fixed_cube(A1, d=100, th=20, name="A1_")
A2_cube = fixed_cube(A2, d=100, th=20, name="A2_")
B1_cube = fixed_cube(B1, d=100, th=20, name="B1_")
B2_cube = fixed_cube(B2, d=100, th=20, name="B2_")

# ---- Model 2 (C vs D) ----
C1_cube = fixed_cube(C1, d=100, th=20, name="C1_")
```

```

C2_cube = fixed_cube(C2, d=100, th=20, name="C2_")
D1_cube = fixed_cube(D1, d=100, th=20, name="D1_")
D2_cube = fixed_cube(D2, d=100, th=20, name="D2_")

# ---- Check tile counts ----
print(f"A tiles: {len(A1_cube) + len(A2_cube)} (A1={len(A1_cube)}, A2={len(A2_cube)})")
print(f"B tiles: {len(B1_cube) + len(B2_cube)} (B1={len(B1_cube)}, B2={len(B2_cube)})")
print(f"C tiles: {len(C1_cube) + len(C2_cube)} (C1={len(C1_cube)}, C2={len(C2_cube)})")
print(f"D tiles: {len(D1_cube) + len(D2_cube)} (D1={len(D1_cube)}, D2={len(D2_cube)})")

```

```

A tiles: 200 (A1=96, A2=104)
B tiles: 211 (B1=107, B2=104)
C tiles: 204 (C1=96, C2=108)
D tiles: 162 (D1=96, D2=66)

```

Observation on tile counts

After tissue filtering at a 20% threshold, the tile counts per class are:

- **Model 1 (A vs B):** A = 200 tiles, B = 211 tiles — well balanced.
- **Model 2 (C vs D):** C = 204 tiles, D = 162 tiles — mild imbalance (~56/44), driven largely by D2's smaller image dimensions producing only 66 tiles.

Both datasets provide a few hundred training examples per class, which is modest for a CNN. The mild imbalance in Model 2 is unlikely to seriously bias training but should be kept in mind when interpreting the results.

```

In [6]: # =====
# LAQ3 - CODE BLOCK 4
# Save tiles to disk
# =====

# Combine cubes for Model 1
AB_cube = {}
AB_cube.update(A1_cube)
AB_cube.update(A2_cube)
AB_cube.update(B1_cube)
AB_cube.update(B2_cube)

# Combine cubes for Model 2
CD_cube = {}
CD_cube.update(C1_cube)
CD_cube.update(C2_cube)
CD_cube.update(D1_cube)
CD_cube.update(D2_cube)

# Save tiles (under the versioned output folder)
save_tiles(AB_cube, versioned_path("cubed_AB"))
save_tiles(CD_cube, versioned_path("cubed_CD"))

```


Saved 411 tiles to outputs_v2/cubed_AB

Saved 366 tiles to outputs_v2/cubed_CD

Inference: The final datasets contain 411 tiles for Model 1 (A/B) and 366 tiles for Model 2 (C/D), confirming a slightly smaller and mildly imbalanced dataset for Model 2.

```
In [7]: # =====
# LAQ3 - CODE BLOCK 5
# Create DataFrames of tile paths and labels
# =====
# Each model needs a table that lists:
# - the tile name
# - the absolute path to the tile on disk
# - the class label (derived from the filename prefix)
# This table is what the next block will use to build train/test split

# ---- Model 1: A vs B ----
tiles_AB = [] # tile names (e.g. "A1_0", "B2_15")
path_AB = [] # absolute file paths pointing to each saved tile
group_AB = [] # class label ("A" or "B"), parsed from the first char

# Loop through every tile we stored in memory for Model 1
for tile in AB_cube:
    tiles_AB.append(tile)
    # Build an absolute path to the saved JPEG under the versioned out
    # Absolute paths are used to avoid any ambiguity about the working
    # when the image loader reads these files later.
    path_AB.append(os.path.abspath(versioned_path("cubed_AB", tile + "
    # The tile name always starts with its source slide letter (A or B
    # so we can recover the class label directly from the filename.
    group_AB.append(tile[0])

# Combine the three lists into a single DataFrame
dat_AB = pd.DataFrame(list(zip(tiles_AB, path_AB, group_AB)),
                        columns=["name", "path", "group"])

# Quick sanity checks:
# - head() confirms paths look correct and labels are parsed properly
# - value_counts() confirms class balance (how many A vs B tiles)
print(dat_AB.head())
print(dat_AB["group"].value_counts())

# ---- Model 2: C vs D ----
# Same logic as above, but for the C vs D dataset.
tiles_CD = []
path_CD = []
group_CD = []

for tile in CD_cube:
    tiles_CD.append(tile)
```



```

path_CD.append(os.path.abspath(versioned_path("cubed_CD", tile + "
# First character of the name is either "C" or "D" – that's our la
group_CD.append(tile[0])

dat_CD = pd.DataFrame(list(zip(tiles_CD, path_CD, group_CD)),
                        columns=["name", "path", "group"])

print(dat_CD.head())
print(dat_CD["group"].value_counts())

```

```

name path group
0  A1_0  /Users/dr.parul/outputs_v2/cubed_AB/A1_0.jpg      A
1  A1_1  /Users/dr.parul/outputs_v2/cubed_AB/A1_1.jpg      A
2  A1_2  /Users/dr.parul/outputs_v2/cubed_AB/A1_2.jpg      A
3  A1_3  /Users/dr.parul/outputs_v2/cubed_AB/A1_3.jpg      A
4  A1_4  /Users/dr.parul/outputs_v2/cubed_AB/A1_4.jpg      A
group
B      211
A      200
Name: count, dtype: int64

```

```

name path group
0  C1_0  /Users/dr.parul/outputs_v2/cubed_CD/C1_0.jpg      C
1  C1_1  /Users/dr.parul/outputs_v2/cubed_CD/C1_1.jpg      C
2  C1_2  /Users/dr.parul/outputs_v2/cubed_CD/C1_2.jpg      C
3  C1_3  /Users/dr.parul/outputs_v2/cubed_CD/C1_3.jpg      C
4  C1_4  /Users/dr.parul/outputs_v2/cubed_CD/C1_4.jpg      C
group
C      204
D      162
Name: count, dtype: int64

```

Observation: DataFrames built successfully with correctly parsed class labels.

The A vs B dataset is well balanced (200 vs 211), while the C vs D dataset shows a mild imbalance (204 vs 162, ~56/44) that should be kept in mind when interpreting Model 2's performance.

```

In [8]: # =====
# Sanity check: confirm tile paths in the DataFrames point to real fil
# =====
# The DataFrames store absolute paths to each saved tile. Before movin
# to training, we verify that those paths actually resolve to files on
# If either check returns False, the image loader in later blocks woul

import os

# Check the first tile path in each DataFrame
print(f"First AB path exists: {os.path.exists(dat_AB['path'].iloc[0])}")
print(f"First CD path exists: {os.path.exists(dat_CD['path'].iloc[0])}")

```

```

First AB path exists: True
First CD path exists: True

```

```
In [9]: # =====
# LAQ3 - CODE BLOCK 6
# Train/Validation split (90:10)
# =====
# The assignment specifies a 90:10 split. The 10% set is used as the
# validation set: it monitors generalization during training (via earl
# stopping) and provides the validation accuracy reported in Q1 and Q2
# A separate held-out test is performed in Q4, where each model is
# applied to the other model's dataset.

from sklearn.model_selection import train_test_split

# ---- Model 1 ----
train_AB, val_AB = train_test_split(
    dat_AB, test_size=0.1, random_state=42,
    stratify=dat_AB["group"]
)
print("Model 1 Train:")
print(train_AB["group"].value_counts())
print("\nModel 1 Validation:")
print(val_AB["group"].value_counts())

# ---- Model 2 ----
train_CD, val_CD = train_test_split(
    dat_CD, test_size=0.1, random_state=42,
    stratify=dat_CD["group"]
)
print("\nModel 2 Train:")
print(train_CD["group"].value_counts())
print("\nModel 2 Validation:")
print(val_CD["group"].value_counts())
```

```

Model 1 Train:
group
B    189
A    180
Name: count, dtype: int64

```

```

Model 1 Validation:
group
B     22
A     20
Name: count, dtype: int64

```

```

Model 2 Train:
group
C     183
D     146
Name: count, dtype: int64

```

```

Model 2 Validation:
group
C     21
D     16
Name: count, dtype: int64

```

Observation: Stratified 90:10 split produced training sets of 369 (Model 1) and 329 (Model 2) tiles, with validation sets of 42 and 37 tiles respectively; class ratios are preserved in both splits as intended.

```

In [10]: # =====
# LAQ3 - CODE BLOCK 7
# Data Generators
# =====
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# ---- Generator settings ----
gtrain = ImageDataGenerator(
    rescale=1.0/255.0,
    vertical_flip=True,
    horizontal_flip=True,
    rotation_range=90,
    height_shift_range=0.2,
    fill_mode="reflect"
)
gval = ImageDataGenerator(rescale=1.0/255.0) # no augmentation on va

BATCH = 16

# =====
# Model 1 (A vs B)
# =====
d_gtrain_AB = gtrain.flow_from_dataframe(
    dataframe=train_AB,

```

```

        x_col="path",
        y_col="group",
        target_size=(100, 100),
        class_mode="categorical",
        batch_size=BATCH,
        shuffle=True
    )
d_gval_AB = gval.flow_from_dataframe(
    dataframe=val_AB,
    x_col="path",
    y_col="group",
    target_size=(100, 100),
    class_mode="categorical",
    batch_size=BATCH,
    shuffle=False
)

# =====
# Model 2 (C vs D)
# =====
d_gtrain_CD = gtrain.flow_from_dataframe(
    dataframe=train_CD,
    x_col="path",
    y_col="group",
    target_size=(100, 100),
    class_mode="categorical",
    batch_size=BATCH,
    shuffle=True
)
d_gval_CD = gval.flow_from_dataframe(
    dataframe=val_CD,
    x_col="path",
    y_col="group",
    target_size=(100, 100),
    class_mode="categorical",
    batch_size=BATCH,
    shuffle=False
)

```

```

/Users/dr.parul/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1
+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(

```

```

Found 369 validated image filenames belonging to 2 classes.
Found 42 validated image filenames belonging to 2 classes.
Found 329 validated image filenames belonging to 2 classes.
Found 37 validated image filenames belonging to 2 classes.

```

Inference Data generators successfully loaded all tiles with correct class labels, consistent with the 90:10 train-validation split, confirming readiness for model training.

```
In [11]: # =====
# Check class indices
# =====
print("Model 1 (A vs B) class indices:")
print(d_gtrain_AB.class_indices)

print("\nModel 2 (C vs D) class indices:")
print(d_gtrain_CD.class_indices)
```

Model 1 (A vs B) class indices:
{'A': 0, 'B': 1}

Model 2 (C vs D) class indices:
{'C': 0, 'D': 1}

Inference: Keras assigned consistent class mappings (A/C → 0, B/D → 1), enabling correct interpretation of predictions during cross-dataset testing.

```
In [25]: # =====
# LAQ3 - CODE BLOCK (Augmentation visualisation)
# =====
# Sanity-check the augmentation pipeline, mirroring the `vis_gen` step
# from the instructor's practical. We pull one batch from each training
# generator and plot the augmented tiles to confirm that flips,
# rotations, and shifts are producing sensible H&E-like images rather
# than artefacts that could mislead the network.

import matplotlib.pyplot as plt

def vis_gen(generator, n=12, title="Augmented batch", ncols=4):
    """
    Plot n augmented tiles from a Keras image generator in a grid.
    Parameters:
        generator: a Keras flow_from_dataframe generator
        n: number of tiles to display
        title: figure title
        ncols: number of columns in the grid
    """
    # Class index -> class name lookup (e.g. {0: 'A', 1: 'B'})
    idx_to_class = {v: k for k, v in generator.class_indices.items()}

    # Pull a single batch from the generator
    X_batch, y_batch = next(generator)

    nrows = (n + ncols - 1) // ncols
    fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 3, nrows *
    axes = axes.flat

    for i in range(n):
        if i >= len(X_batch):
            break # safety check if batch is smaller than n
```

```

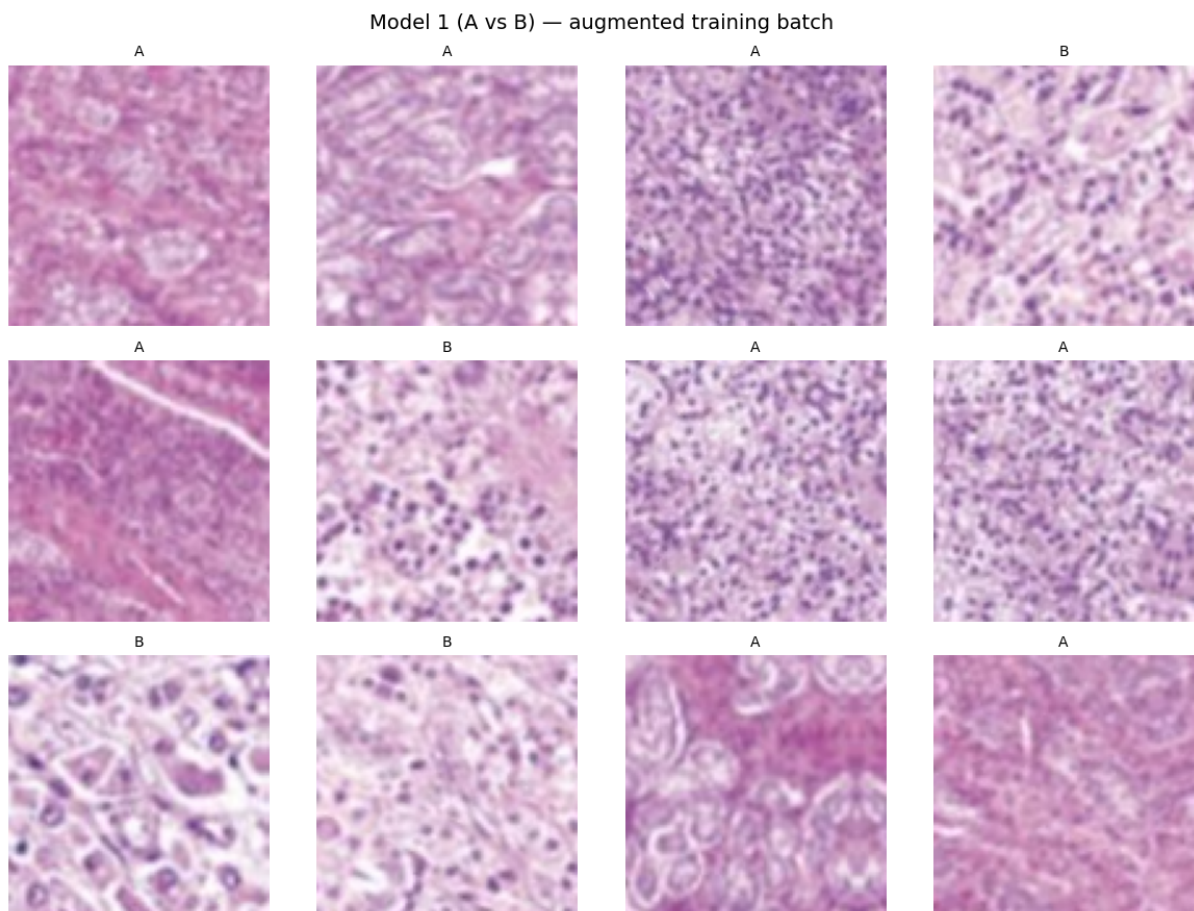
axes[i].imshow(X_batch[i])
# Recover label from one-hot encoding
label = idx_to_class[int(y_batch[i].argmax())]
axes[i].set_title(f"{label}", fontsize=10)
axes[i].axis("off")

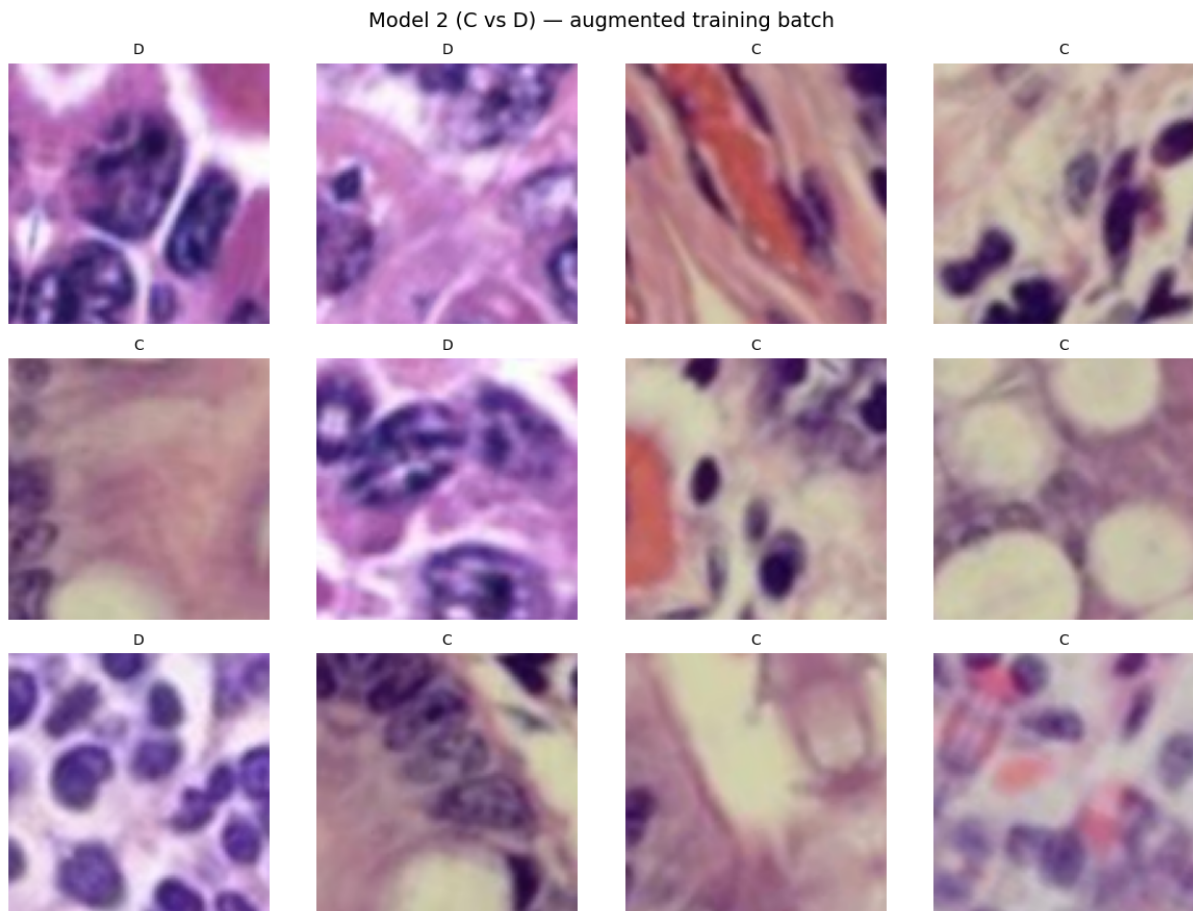
# Hide any unused axes
for j in range(n, len(axes)):
    axes[j].axis("off")

plt.suptitle(title, fontsize=14)
plt.tight_layout()
plt.show()

# Visualise one augmented batch from each training generator
vis_gen(d_gtrain_AB, n=12, title="Model 1 (A vs B) — augmented training batch")
vis_gen(d_gtrain_CD, n=12, title="Model 2 (C vs D) — augmented training batch")

```





Inference: Augmentation is producing varied orientations and crops of valid H&E tissue with correct class labels, confirming the pipeline is working as intended.

```
In [13]: # =====
# LAQ3 - CODE BLOCK 8
# CNN Model Definitions
# =====

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

# Reproducibility: ensures same results across runs (important for rep
import numpy as np
import tensorflow as tf
np.random.seed(42)
tf.random.set_seed(42)

def define_model_1():
    """
    Model 1 (A vs B): 3 Conv2D blocks.
    """
    model = Sequential()
```



```

# Block 1
# 'same' padding preserves spatial dimensions → better for small 1
model.add(Conv2D(32, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same',
                 input_shape=(100, 100, 3)))
model.add(MaxPooling2D((2, 2))) # downsample
model.add(BatchNormalization()) # stabilise training

# Block 2
model.add(Conv2D(64, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same'))
model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Block 3
model.add(Conv2D(128, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same'))
model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Classifier head
model.add(Flatten())
model.add(Dropout(0.4)) # reduce overfitting (small dataset)

# Slightly wider layer improves representation capacity (resource-
model.add(Dense(128, activation='relu', kernel_initializer='he_uni
model.add(Dropout(0.3))

model.add(Dense(2, activation='softmax')) # binary classification

model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
return model

def define_model_2():
    """
    Model 2 (C vs D): 4 Conv2D blocks – deeper architecture
    with lower learning rate for stability.
    """
    model = Sequential()

    # Block 1
    model.add(Conv2D(32, (3, 3), activation='relu',
                     kernel_initializer='he_uniform',

```

```

        padding='same',
        input_shape=(100, 100, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Block 2
model.add(Conv2D(64, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same'))
model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Block 3
model.add(Conv2D(128, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same'))
model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Block 4 (extra depth)
model.add(Conv2D(256, (3, 3), activation='relu',
                 kernel_initializer='he_uniform',
                 padding='same'))
model.add(BatchNormalization())

# Classifier head
model.add(Flatten())
model.add(Dropout(0.5)) # stronger regularisation for deeper models

model.add(Dense(128, activation='relu', kernel_initializer='he_uniform'))
model.add(Dropout(0.3))

model.add(Dense(2, activation='softmax'))

model.compile(
    optimizer=Adam(learning_rate=0.0003), # lower LR for stability
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
return model

print("Model definitions ready.")

```

Model definitions ready.

```

In [15]: # =====
# LAQ3 - CODE BLOCK 9
# Train Model 1 (A vs B)
# =====
# This block trains the first CNN to classify A vs B tiles.
# Two callbacks control the training process: early stopping (to
# prevent overfitting) and model checkpointing (to preserve the

```

```

# best weights for later reuse in Q4 cross-testing).

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Maximum number of training epochs. Early stopping will usually
# halt training before this is reached on a dataset of this size.
EPOCHS = 30

# Early stopping callback:
# - monitors validation loss each epoch
# - if val_loss does not improve for 7 consecutive epochs, training
# - restore_best_weights=True rolls the model back to the epoch with
#   lowest validation loss, so the final model is the best-generalis
#   version rather than the last (potentially overfit) one
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True,
    verbose=1
)

# Model checkpoint callback:
# - saves the model to disk whenever validation loss reaches a new m
# - save_best_only=True ensures we keep only the single best version
#   rather than one file per epoch
# - this gives us a saved artefact we can reload later for Q4
#   cross-testing without retraining
checkpoint_AB = ModelCheckpoint(
    'model_AB_best.keras',
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

# Instantiate the Model 1 architecture (3 Conv2D blocks, defined earli
model_AB = define_model_1()

# Print the layer-by-layer summary so we can confirm the architecture
# and the parameter count before training
model_AB.summary()

# Train the model. The training generator feeds in batches of 16 augme
# tiles (flips, rotations, shifts) while the validation generator is
# rescaled-only with no augmentation, providing a clean estimate of
# generalisation performance.
history_AB = model_AB.fit(
    d_gtrain_AB,          # augmented training generator
    epochs=EPOCHS,        # cap on epochs (early stop usually
    validation_data=d_gval_AB, # validation generator for monitorin
    callbacks=[early_stop, checkpoint_AB],
    verbose=1
)

```

Model: "sequential_1"

Layer (type)	Output Shape	
conv2d_3 (Conv2D)	(None, 100, 100, 32)	
max_pooling2d_3 (MaxPooling2D)	(None, 50, 50, 32)	
batch_normalization_3 (BatchNormalization)	(None, 50, 50, 32)	
conv2d_4 (Conv2D)	(None, 50, 50, 64)	
max_pooling2d_4 (MaxPooling2D)	(None, 25, 25, 64)	
batch_normalization_4 (BatchNormalization)	(None, 25, 25, 64)	
conv2d_5 (Conv2D)	(None, 25, 25, 128)	
max_pooling2d_5 (MaxPooling2D)	(None, 12, 12, 128)	
batch_normalization_5 (BatchNormalization)	(None, 12, 12, 128)	
flatten_1 (Flatten)	(None, 18432)	
dropout_2 (Dropout)	(None, 18432)	
dense_2 (Dense)	(None, 128)	
dropout_3 (Dropout)	(None, 128)	
dense_3 (Dense)	(None, 2)	

Total params: 2,453,826 (9.36 MB)

Trainable params: 2,453,378 (9.36 MB)

Non-trainable params: 448 (1.75 KB)

Epoch 1/30

24/24 ————— 0s 36ms/step - accuracy: 0.7397 - loss: 1.3723

Epoch 1: val_loss improved from inf to 11.87541, saving model to model_AB_best.keras

24/24 ————— 2s 44ms/step - accuracy: 0.7423 - loss: 1.3585 - val_accuracy: 0.5238 - val_loss: 11.8754

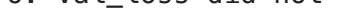
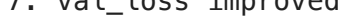
Epoch 2/30

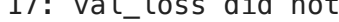
24/24 ————— 0s 36ms/step - accuracy: 0.8222 - loss: 0.5919

Epoch 2: val_loss improved from 11.87541 to 4.14886, saving model to model_AB_best.keras

24/24 ————— 1s 39ms/step - accuracy: 0.8229 - loss: 0.5883 - val_accuracy: 0.5238 - val_loss: 4.1489

Epoch 3/30

23/24  0s 37ms/step - accuracy: 0.8660 - loss: 0.3123
Epoch 3: val_loss improved from 4.14886 to 0.44980, saving model to model_AB_best.keras
24/24  1s 38ms/step - accuracy: 0.8661 - loss: 0.3157 - val_accuracy: 0.7857 - val_loss: 0.4498
Epoch 4/30
23/24  0s 36ms/step - accuracy: 0.8957 - loss: 0.2757
Epoch 4: val_loss did not improve from 0.44980
24/24  1s 38ms/step - accuracy: 0.8951 - loss: 0.2761 - val_accuracy: 0.6190 - val_loss: 0.6641
Epoch 5/30
24/24  0s 36ms/step - accuracy: 0.8785 - loss: 0.3115
Epoch 5: val_loss did not improve from 0.44980
24/24  1s 38ms/step - accuracy: 0.8790 - loss: 0.3103 - val_accuracy: 0.4762 - val_loss: 0.8606
Epoch 6/30
24/24  0s 36ms/step - accuracy: 0.9051 - loss: 0.2783
Epoch 6: val_loss did not improve from 0.44980
24/24  1s 38ms/step - accuracy: 0.9053 - loss: 0.2787 - val_accuracy: 0.4762 - val_loss: 0.8768
Epoch 7/30
23/24  0s 37ms/step - accuracy: 0.8932 - loss: 0.2155
Epoch 7: val_loss improved from 0.44980 to 0.31410, saving model to model_AB_best.keras
24/24  1s 39ms/step - accuracy: 0.8924 - loss: 0.2175 - val_accuracy: 0.9048 - val_loss: 0.3141
Epoch 8/30
23/24  0s 38ms/step - accuracy: 0.9334 - loss: 0.1714
Epoch 8: val_loss did not improve from 0.31410
24/24  1s 39ms/step - accuracy: 0.9320 - loss: 0.1754 - val_accuracy: 0.4762 - val_loss: 0.8667
Epoch 9/30
24/24  0s 38ms/step - accuracy: 0.9591 - loss: 0.1596
Epoch 9: val_loss did not improve from 0.31410
24/24  1s 39ms/step - accuracy: 0.9587 - loss: 0.1592 - val_accuracy: 0.4762 - val_loss: 1.6998
Epoch 10/30
23/24  0s 37ms/step - accuracy: 0.9198 - loss: 0.1586
Epoch 10: val_loss did not improve from 0.31410
24/24  1s 38ms/step - accuracy: 0.9208 - loss: 0.1576 - val_accuracy: 0.5238 - val_loss: 1.0974
Epoch 11/30
23/24  0s 38ms/step - accuracy: 0.9355 - loss: 0.1362

Epoch 11: val_loss did not improve from 0.31410
24/24  1s 39ms/step - accuracy: 0.9353 - loss: 0.1369 - val_accuracy: 0.4762 - val_loss: 2.1117
Epoch 12/30
24/24  0s 38ms/step - accuracy: 0.9461 - loss: 0.1377
Epoch 12: val_loss did not improve from 0.31410
24/24  1s 40ms/step - accuracy: 0.9459 - loss: 0.1402 - val_accuracy: 0.4762 - val_loss: 1.9639
Epoch 13/30
23/24  0s 38ms/step - accuracy: 0.9462 - loss: 0.1355
Epoch 13: val_loss improved from 0.31410 to 0.18120, saving model to model_AB_best.keras
24/24  1s 40ms/step - accuracy: 0.9444 - loss: 0.1381 - val_accuracy: 0.9524 - val_loss: 0.1812
Epoch 14/30
23/24  0s 39ms/step - accuracy: 0.9521 - loss: 0.0998
Epoch 14: val_loss did not improve from 0.18120
24/24  1s 40ms/step - accuracy: 0.9525 - loss: 0.1021 - val_accuracy: 0.9048 - val_loss: 0.2012
Epoch 15/30
23/24  0s 38ms/step - accuracy: 0.9493 - loss: 0.1257
Epoch 15: val_loss improved from 0.18120 to 0.11342, saving model to model_AB_best.keras
24/24  1s 41ms/step - accuracy: 0.9493 - loss: 0.1257 - val_accuracy: 0.9524 - val_loss: 0.1134
Epoch 16/30
23/24  0s 38ms/step - accuracy: 0.9524 - loss: 0.1049
Epoch 16: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9517 - loss: 0.1082 - val_accuracy: 0.9286 - val_loss: 0.1367
Epoch 17/30
24/24  0s 38ms/step - accuracy: 0.9432 - loss: 0.1218
Epoch 17: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9437 - loss: 0.1208 - val_accuracy: 0.9286 - val_loss: 0.1777
Epoch 18/30
24/24  0s 39ms/step - accuracy: 0.9247 - loss: 0.1806
Epoch 18: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9239 - loss: 0.1819 - val_accuracy: 0.9286 - val_loss: 0.1961
Epoch 19/30
24/24  0s 39ms/step - accuracy: 0.9386 - loss: 0.2303
Epoch 19: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9375 - loss: 0.23

11 - val_accuracy: 0.8571 - val_loss: 0.1672
Epoch 20/30
23/24  0s 38ms/step - accuracy: 0.9463 - loss: 0.1654
Epoch 20: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9467 - loss: 0.1661 - val_accuracy: 0.7857 - val_loss: 2.4928
Epoch 21/30
24/24  0s 39ms/step - accuracy: 0.9642 - loss: 0.0849
Epoch 21: val_loss did not improve from 0.11342
24/24  1s 40ms/step - accuracy: 0.9641 - loss: 0.0852 - val_accuracy: 0.8095 - val_loss: 1.4328
Epoch 22/30
23/24  0s 40ms/step - accuracy: 0.9493 - loss: 0.1309
Epoch 22: val_loss improved from 0.11342 to 0.09546, saving model to model_AB_best.keras
24/24  1s 43ms/step - accuracy: 0.9501 - loss: 0.1297 - val_accuracy: 0.9762 - val_loss: 0.0955
Epoch 23/30
23/24  0s 40ms/step - accuracy: 0.9454 - loss: 0.1865
Epoch 23: val_loss improved from 0.09546 to 0.03046, saving model to model_AB_best.keras
24/24  1s 42ms/step - accuracy: 0.9459 - loss: 0.1849 - val_accuracy: 0.9762 - val_loss: 0.0305
Epoch 24/30
24/24  0s 40ms/step - accuracy: 0.9529 - loss: 0.1894
Epoch 24: val_loss did not improve from 0.03046
24/24  1s 42ms/step - accuracy: 0.9527 - loss: 0.1894 - val_accuracy: 0.9524 - val_loss: 0.1426
Epoch 25/30
24/24  0s 39ms/step - accuracy: 0.9542 - loss: 0.0967
Epoch 25: val_loss did not improve from 0.03046
24/24  1s 41ms/step - accuracy: 0.9543 - loss: 0.0968 - val_accuracy: 0.9286 - val_loss: 0.1649
Epoch 26/30
24/24  0s 40ms/step - accuracy: 0.9671 - loss: 0.1019
Epoch 26: val_loss did not improve from 0.03046
24/24  1s 42ms/step - accuracy: 0.9668 - loss: 0.1026 - val_accuracy: 0.6905 - val_loss: 4.8038
Epoch 27/30
23/24  0s 39ms/step - accuracy: 0.9222 - loss: 0.1513
Epoch 27: val_loss did not improve from 0.03046
24/24  1s 41ms/step - accuracy: 0.9237 - loss: 0.1513 - val_accuracy: 0.7143 - val_loss: 3.2187
Epoch 28/30


```

23/24 ————— 0s 39ms/step - accuracy: 0.9393 - loss: 0.15
80
Epoch 28: val_loss did not improve from 0.03046
24/24 ————— 1s 41ms/step - accuracy: 0.9406 - loss: 0.15
53 - val_accuracy: 0.5476 - val_loss: 12.6291
Epoch 29/30
23/24 ————— 0s 39ms/step - accuracy: 0.9500 - loss: 0.12
69
Epoch 29: val_loss did not improve from 0.03046
24/24 ————— 1s 41ms/step - accuracy: 0.9518 - loss: 0.12
56 - val_accuracy: 0.5238 - val_loss: 16.6006
Epoch 30/30
23/24 ————— 0s 41ms/step - accuracy: 0.9828 - loss: 0.04
42
Epoch 30: val_loss did not improve from 0.03046
24/24 ————— 1s 42ms/step - accuracy: 0.9818 - loss: 0.04
77 - val_accuracy: 0.8333 - val_loss: 1.2767
Epoch 30: early stopping
Restoring model weights from the end of the best epoch: 23.

```

Answer 1: Observation on Model 1 training Model 1 (A vs B) trained for 30 epochs before early stopping triggered, with weights restored from the best epoch (23). Training accuracy increased steadily to approximately 94–98%, while validation accuracy showed substantial variability due to the small validation set (42 tiles), where a single misclassification changes accuracy by ~2.4 percentage points. The best validation performance was achieved at epoch 23 (97.6%, val_loss = 0.0305). Despite broadly similar stain profiles between groups A and B, the model was able to learn discriminative features, although fluctuations in validation performance reflect sampling variability and potential overfitting beyond the optimal epoch.

```

In [16]: # =====
# LAQ3 - CODE BLOCK 10
# Train Model 2 (C vs D)
# =====

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Early stopping: stops training when validation loss stops improving
# and restores the best-performing weights
early_stop2 = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True,
    verbose=1
)

# Model checkpoint: saves the best model (lowest validation loss)
# so it can be reused later for Q4 cross-testing

```

```
checkpoint_CD = ModelCheckpoint(  
    'model_CD_best.keras',  
    monitor='val_loss',  
    save_best_only=True,  
    verbose=1  
)  
  
# Instantiate Model 2 (deeper CNN with 4 Conv blocks)  
model_CD = define_model_2()  
  
# Print architecture summary to verify structure and parameter count  
model_CD.summary()  
  
# Train the model using training generator with augmentation  
# and validation generator without augmentation  
history_CD = model_CD.fit(  
    d_gtrain_CD,  
    epochs=EPOCHS,  
    validation_data=d_gval_CD,  
    callbacks=[early_stop2, checkpoint_CD],  
    verbose=1  
)  
  
# Save final model (best weights already restored via early stopping)  
model_CD.save('model_CD_final.keras')
```

Model: "sequential_2"

Layer (type)	Output Shape	
conv2d_6 (Conv2D)	(None, 100, 100, 32)	
max_pooling2d_6 (MaxPooling2D)	(None, 50, 50, 32)	
batch_normalization_6 (BatchNormalization)	(None, 50, 50, 32)	
conv2d_7 (Conv2D)	(None, 50, 50, 64)	
max_pooling2d_7 (MaxPooling2D)	(None, 25, 25, 64)	
batch_normalization_7 (BatchNormalization)	(None, 25, 25, 64)	
conv2d_8 (Conv2D)	(None, 25, 25, 128)	
max_pooling2d_8 (MaxPooling2D)	(None, 12, 12, 128)	
batch_normalization_8 (BatchNormalization)	(None, 12, 12, 128)	
conv2d_9 (Conv2D)	(None, 12, 12, 256)	
batch_normalization_9 (BatchNormalization)	(None, 12, 12, 256)	
flatten_2 (Flatten)	(None, 36864)	
dropout_4 (Dropout)	(None, 36864)	
dense_4 (Dense)	(None, 128)	
dropout_5 (Dropout)	(None, 128)	
dense_5 (Dense)	(None, 2)	

Total params: 5,109,314 (19.49 MB)

Trainable params: 5,108,354 (19.49 MB)

Non-trainable params: 960 (3.75 KB)

Epoch 1/30

21/21 ————— 0s 49ms/step – accuracy: 0.8872 – loss: 0.3568

Epoch 1: val_loss improved from inf to 4.77016, saving model to model_C D_best.keras

21/21 ————— 3s 59ms/step – accuracy: 0.8908 – loss: 0.3459 – val_accuracy: 0.4324 – val_loss: 4.7702

Epoch 2/30

20/21 ————— 0s 49ms/step – accuracy: 0.9907 – loss: 0.0795

Epoch 2: val_loss did not improve from 4.77016

21/21  **1s** 51ms/step - accuracy: 0.9907 - loss: 0.0782 - val_accuracy: 0.4324 - val_loss: 4.8798
Epoch 3/30

21/21  **0s** 49ms/step - accuracy: 1.0000 - loss: 0.0000e+00
Epoch 3: val_loss improved from 4.77016 to 0.37187, saving model to model_CD_best.keras

21/21  **1s** 53ms/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 0.8108 - val_loss: 0.3719
Epoch 4/30

21/21  **0s** 50ms/step - accuracy: 1.0000 - loss: 7.5063e-04
Epoch 4: val_loss improved from 0.37187 to 0.00002, saving model to model_CD_best.keras

21/21  **1s** 54ms/step - accuracy: 1.0000 - loss: 7.5654e-04 - val_accuracy: 1.0000 - val_loss: 2.1096e-05
Epoch 5/30

20/21  **0s** 50ms/step - accuracy: 1.0000 - loss: 6.8723e-09
Epoch 5: val_loss improved from 0.00002 to 0.00000, saving model to model_CD_best.keras

21/21  **1s** 54ms/step - accuracy: 1.0000 - loss: 7.3675e-09 - val_accuracy: 1.0000 - val_loss: 8.8600e-07
Epoch 6/30

20/21  **0s** 52ms/step - accuracy: 1.0000 - loss: 4.1086e-10
Epoch 6: val_loss did not improve from 0.00000

21/21  **1s** 53ms/step - accuracy: 1.0000 - loss: 5.7114e-10 - val_accuracy: 1.0000 - val_loss: 8.8493e-05
Epoch 7/30

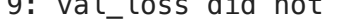
20/21  **0s** 50ms/step - accuracy: 0.9884 - loss: 0.4215
Epoch 7: val_loss did not improve from 0.00000

21/21  **1s** 52ms/step - accuracy: 0.9850 - loss: 0.4853 - val_accuracy: 0.5676 - val_loss: 8.7745
Epoch 8/30

21/21  **0s** 50ms/step - accuracy: 0.9662 - loss: 0.0837
Epoch 8: val_loss did not improve from 0.00000

21/21  **1s** 52ms/step - accuracy: 0.9668 - loss: 0.0840 - val_accuracy: 0.7297 - val_loss: 1.5120
Epoch 9/30

20/21  **0s** 51ms/step - accuracy: 0.9889 - loss: 0.0813
Epoch 9: val_loss did not improve from 0.00000

21/21  **1s** 53ms/step - accuracy: 0.9891 - loss: 0.0801 - val_accuracy: 0.6757 - val_loss: 2.9333
Epoch 10/30

21/21  **0s** 51ms/step - accuracy: 0.9996 - loss: 0.0091
Epoch 10: val_loss did not improve from 0.00000

21/21  **1s** 53ms/step - accuracy: 0.9992 - loss: 0.01

```

74 - val_accuracy: 0.7297 - val_loss: 2.2583
Epoch 11/30
21/21 ————— 0s 52ms/step - accuracy: 1.0000 - loss: 0.00
00e+00
Epoch 11: val_loss did not improve from 0.00000
21/21 ————— 1s 54ms/step - accuracy: 1.0000 - loss: 0.00
00e+00 - val_accuracy: 1.0000 - val_loss: 5.5090e-06
Epoch 12/30
21/21 ————— 0s 52ms/step - accuracy: 0.9950 - loss: 0.02
42
Epoch 12: val_loss did not improve from 0.00000
21/21 ————— 1s 55ms/step - accuracy: 0.9949 - loss: 0.02
45 - val_accuracy: 0.9189 - val_loss: 0.3068
Epoch 12: early stopping
Restoring model weights from the end of the best epoch: 5.

```

Answer 2 Observation on Model 2 training Model 2 (C vs D) converged extremely rapidly, achieving near-perfect training accuracy (~100%) and peak validation accuracy of 100% by epoch 5, after which early stopping restored the best weights. However, validation performance showed significant instability beyond this point, with accuracy dropping sharply in later epochs. Given the small validation set (37 tiles), where a single misclassification changes accuracy by ~2.7 percentage points, the observed perfect accuracy is likely inflated. Additionally, the model contains over 5 million parameters relative to a few hundred training samples, indicating substantial overfitting. The perfect validation accuracy is consistent with the model exploiting a low-level cue — most plausibly a global stain or colour shift between the C and D source images — rather than learning generalisable morphological features. This will be revisited in Q4 and Q5.

```

In [17]: # =====
# LAQ3 - CODE BLOCK 11
# Evaluate Model 1 and Model 2 (answers Q1 and Q2)
# =====
# model.evaluate() reports the loss and accuracy on a given generator
# without applying any augmentation. We evaluate on both the training
# and validation sets so we can report both numbers for Q1 and Q2.

# -----
# Model 1 (A vs B)
# -----
# Build a no-augmentation generator over the training set for a clean
# training accuracy measurement (the per-epoch accuracy printed during
# fit() is across augmented batches, which slightly underestimates
# the model's accuracy on the real training data).
d_geval_train_AB = gval.flow_from_dataframe(
    dataframe=train_AB,
    x_col="path",
    y_col="group",
    target_size=(100, 100),

```

```

        class_mode="categorical",
        batch_size=BATCH,
        shuffle=False
    )

    train_score_AB = model_AB.evaluate(d_geval_train_AB, verbose=0)
    val_score_AB = model_AB.evaluate(d_gval_AB, verbose=0)

    print("=== Model 1 (A vs B) ===")
    print(f"Training accuracy:  {train_score_AB[1]:.4f} (loss: {train_sc
    print(f"Validation accuracy: {val_score_AB[1]:.4f} (loss: {val_score_

# -----
# Model 2 (C vs D)
# -----
d_geval_train_CD = gval.flow_from_dataframe(
    dataframe=train_CD,
    x_col="path",
    y_col="group",
    target_size=(100, 100),
    class_mode="categorical",
    batch_size=BATCH,
    shuffle=False
)

train_score_CD = model_CD.evaluate(d_geval_train_CD, verbose=0)
val_score_CD = model_CD.evaluate(d_gval_CD, verbose=0)

print("\n=== Model 2 (C vs D) ===")
print(f"Training accuracy:  {train_score_CD[1]:.4f} (loss: {train_sc
print(f"Validation accuracy: {val_score_CD[1]:.4f} (loss: {val_score_

```

Found 369 validated image filenames belonging to 2 classes.

/Users/dr.parul/Library/Python/3.9/lib/python/site-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

self._warn_if_super_not_called()

=== Model 1 (A vs B) ===

Training accuracy: 0.9675 (loss: 0.0834)

Validation accuracy: 0.9762 (loss: 0.0305)

Found 329 validated image filenames belonging to 2 classes.

/Users/dr.parul/Library/Python/3.9/lib/python/site-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

self._warn_if_super_not_called()

```

=== Model 2 (C vs D) ===
Training accuracy:  1.0000  (loss: 0.0000)
Validation accuracy: 1.0000  (loss: 0.0000)

```

Results for Q1 and Q2

Model 1 (A vs B):

Training accuracy: 96.75% (loss 0.0834)

Validation accuracy: 97.62% (loss 0.0305)

Model 2 (C vs D):

Training accuracy: 100.00% (loss 0.0000)

Validation accuracy: 100.00% (loss 0.0000)

Both models achieved strong apparent performance on their respective datasets. Model 2 reached perfect validation accuracy, slightly exceeding Model 1 (97.6%). However, the learning dynamics differed markedly: Model 1 showed gradual convergence over multiple epochs, whereas Model 2 converged extremely rapidly, achieving near-perfect performance within a few epochs. Given the small validation set sizes and the pronounced stain differences observed between groups C and D, Model 2's perfect accuracy is likely driven by easily separable low-level features rather than robust generalisable representations.

```

In [18]: # =====
# LAQ3 - CODE BLOCK 12
# Q4: Cross-dataset testing
# =====
# Each model is applied to the other dataset's validation set.
# Because the generators assign class indices alphabetically
# (A->0, B->1 for Model 1; C->0, D->1 for Model 2), the reported
# "accuracy" reflects how often each model's output class index
# matches the other dataset's true class index.
# The interpretation of these numbers (and the reasons behind them)
# is addressed in Q5.

# Model 1 evaluated on the Model 2 validation set (C vs D tiles)
score_AB_on_CD = model_AB.evaluate(d_gval_CD, verbose=0)
print("Model 1 applied to C/D data:")
print(f"  Loss:      {score_AB_on_CD[0]:.4f}")
print(f"  Accuracy: {score_AB_on_CD[1]:.4f}")

# Model 2 evaluated on the Model 1 validation set (A vs B tiles)
score_CD_on_AB = model_CD.evaluate(d_gval_AB, verbose=0)
print("\nModel 2 applied to A/B data:")
print(f"  Loss:      {score_CD_on_AB[0]:.4f}")

```



```
print(f" Accuracy: {score_CD_on_AB[1]:.4f}")
```

Model 1 applied to C/D data:

Loss: 57.4434

Accuracy: 0.4595

Model 2 applied to A/B data:

Loss: 8.9207

Accuracy: 0.5238

```
In [22]: # =====
# Training History Plots
# =====

import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

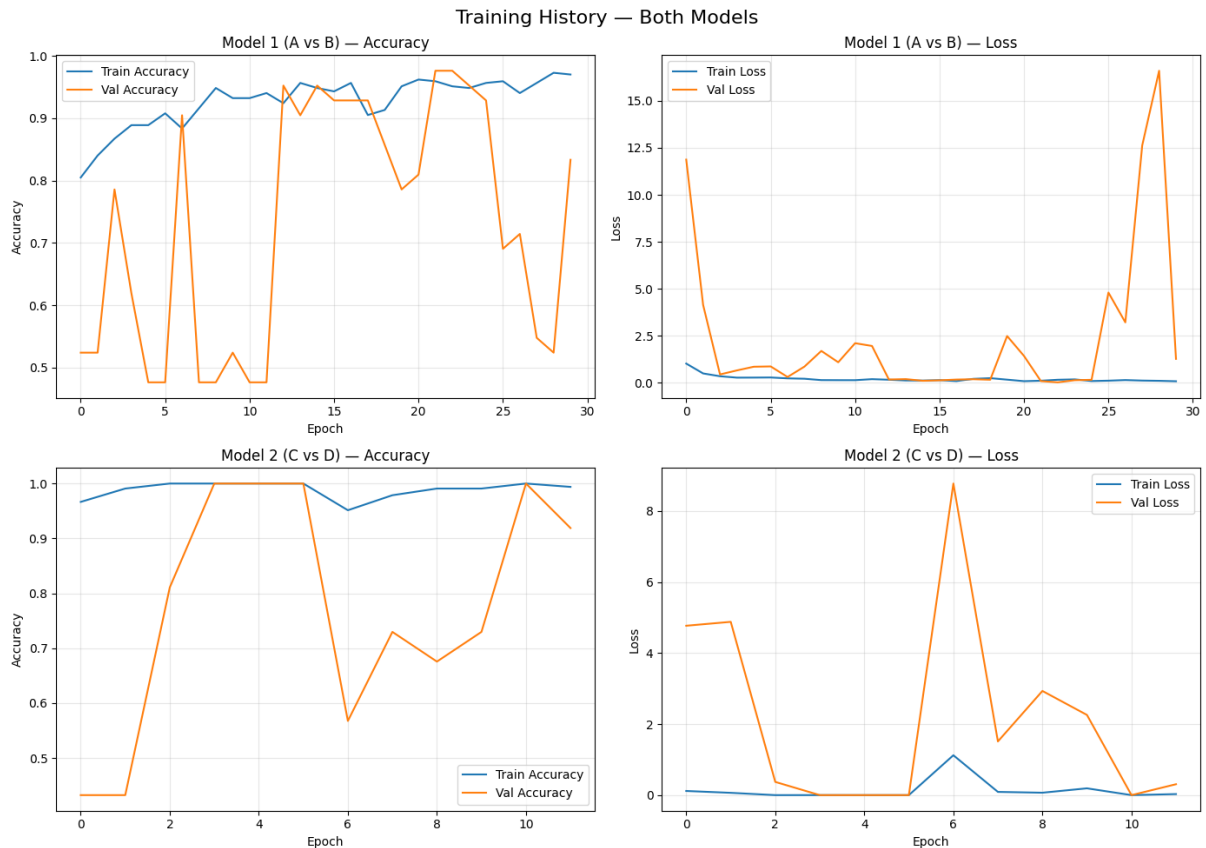
# Model 1 - Accuracy
axes[0, 0].plot(history_AB.history['accuracy'], label='Train Accuracy')
axes[0, 0].plot(history_AB.history['val_accuracy'], label='Val Accuracy')
axes[0, 0].set_title('Model 1 (A vs B) - Accuracy')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Model 1 - Loss
axes[0, 1].plot(history_AB.history['loss'], label='Train Loss')
axes[0, 1].plot(history_AB.history['val_loss'], label='Val Loss')
axes[0, 1].set_title('Model 1 (A vs B) - Loss')
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Loss')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Model 2 - Accuracy
axes[1, 0].plot(history_CD.history['accuracy'], label='Train Accuracy')
axes[1, 0].plot(history_CD.history['val_accuracy'], label='Val Accuracy')
axes[1, 0].set_title('Model 2 (C vs D) - Accuracy')
axes[1, 0].set_xlabel('Epoch')
axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Model 2 - Loss
axes[1, 1].plot(history_CD.history['loss'], label='Train Loss')
axes[1, 1].plot(history_CD.history['val_loss'], label='Val Loss')
axes[1, 1].set_title('Model 2 (C vs D) - Loss')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Loss')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)
```

```
plt.suptitle('Training History — Both Models', fontsize=16)
plt.tight_layout()
plt.show()
```



Inference The training curves further illustrate the contrast: Model 1 shows gradual convergence over approximately 15 epochs, with substantial variability in validation accuracy reflecting the small validation set. In contrast, Model 2 reaches near-perfect training accuracy within the first few epochs, with training loss rapidly collapsing to near zero. This behaviour is consistent with the network exploiting a low-level cue early in training, rather than progressively learning more complex and generalisable discriminative features.

Q3. Which of the models performed best on its dataset, and possible reasons for the differences in performance

On its own dataset, **Model 2 (C vs D)** outperformed Model 1 (A vs B), achieving 100% training and validation accuracy compared with 96.75% and 97.62% for Model 1. However, this apparent superiority is unlikely to reflect genuinely better feature learning. Three factors plausibly explain the difference:

1. Class separability

Direct inspection of the source images (displayed above in this file) reveals a

fundamental difference in the type of cue available to each model. Groups A and B share a broadly similar global stain profile — both predominantly eosinophilic pink with interspersed basophilic regions — but differ at the architectural level. For example, A1 contains a lymphoid follicle adjacent to glandular and stromal tissue, A2 is a relatively uniform cellular field, B1 includes a distinct region of adipose tissue alongside glandular structures, and B2 appears more diffusely cellular. Separating A from B therefore requires the network to learn higher-order texture and structural features rather than relying on global colour.

In contrast, groups C and D differ overtly at the colour and cellular density level. C samples show typical glandular epithelium with mixed pink and purple staining, whereas D samples are densely cellular and strongly basophilic, dominated by small nuclei (with D2 in particular appearing almost entirely hematoxylin-stained). While this reflects a genuine biological difference in tissue composition, it also provides a near-trivial low-level cue that a convolutional neural network can exploit without learning meaningful morphology. Training loss for Model 2 reached zero by epoch 3, consistent with the model rapidly locking onto this global colour and density signal, whereas Model 1 converged more slowly due to the absence of such a shortcut. This behaviour is consistent with prior work showing that CNNs preferentially learn stain-related features (Tellez et al., 2019) and can exploit site- or preparation-specific signatures (Howard et al., 2021).

2. Model capacity vs data size

Model 2 contains approximately 5.1 million parameters trained on ~329 tiles, yielding a very high parameter-to-sample ratio that strongly favours overfitting. Model 1 (~2.45 million parameters) is also over-parameterised, but to a lesser extent. As highlighted by Janowczyk and Madabhushi (2016), limited data combined with variability in staining and slide preparation makes digital pathology particularly susceptible to overfitting, with models often capturing dataset-specific artefacts rather than generalisable biological signals.

3. Validation set size

The validation sets are small (37 tiles for Model 2 and 42 for Model 1), meaning a single misclassification shifts accuracy by approximately 2.5–2.7 percentage points. Consequently, reported validation accuracies have wide confidence intervals, and the difference between 100% and 97.62% should not be over-interpreted.

Conclusion

The cross-dataset results in Q4 are consistent with this interpretation: despite

perfect within-domain performance, Model 2 collapses to near-random accuracy when applied to A/B data, where the dominant colour cue distinguishing C from D is absent. This indicates reliance on dataset-specific features rather than robust, generalisable morphological representations.

References

1. Howard, F. M., Dolezal, J., Kochanny, S., et al. (2021). The impact of site-specific digital histology signatures on deep learning model accuracy and bias. *Nature Communications*, 12, 4423. <https://doi.org/10.1038/s41467-021-24698-1>
2. Janowczyk, A. and Madabhushi, A. (2016). Deep learning for digital pathology image analysis: A comprehensive tutorial with selected use cases. *Journal of Pathology Informatics*, 7, 29. <https://doi.org/10.4103/2153-3539.186902>
3. Tellez, D., Litjens, G., Bándi, P., et al. (2019). Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology. *Medical Image Analysis*, 58, 101544. <https://doi.org/10.1016/j.media.2019.101544>

```
In [21]: # =====
# Detailed cross-test analysis (for Q4/Q5 interpretation)
# =====
# This block goes beyond reporting cross-test accuracy by examining
# *how* each model is making predictions on the other dataset.
# Specifically, it counts how many tiles from each true class are
# assigned to each predicted class. This helps determine whether
# cross-dataset failure is:
# (i) random (~50/50 predictions), or
# (ii) systematic (e.g. bias toward a single class),
# which provides insight into whether the model has learned
# generalisable morphological features or dataset-specific cues
# (such as stain colour or intensity).

import numpy as np

print("\n" + "="*60)
print("Prediction distribution (cross-dataset)")
print("="*60)

# -----
# Model 1 (A vs B) applied to C/D tiles
# -----
# Run the trained Model 1 over every C/D validation tile to get
# softmax probabilities for the (A, B) classes.
preds_AB_on_CD = model_AB.predict(d_gval_CD, verbose=0)
```

```

# Convert the 2-column softmax output to a single predicted class
# index per tile (0 = "A-like", 1 = "B-like").
pred_classes_AB_on_CD = np.argmax(preds_AB_on_CD, axis=1)

# Pull the true class labels from the generator. Wrapping in np.array(
# is necessary because Keras stores .classes as a Python list, which
# does not support element-wise comparison via the (== 0) / (== 1)
# operators used below.
true_classes_CD = np.array(d_gval_CD.classes) # 0 = C, 1 = D

# For each true class (C or D), count how many tiles were predicted
# as A-like (0) versus B-like (1). A strongly skewed distribution
# would indicate the model is defaulting to one class rather than
# making a calibrated decision.
print("\nModel 1 (A vs B) applied to C/D tiles:")
print(f" True C tiles ({np.sum(true_classes_CD == 0)}): "
      f"predicted class 0 = {np.sum((true_classes_CD == 0) & (pred_cla
      f"predicted class 1 = {np.sum((true_classes_CD == 0) & (pred_cla
print(f" True D tiles ({np.sum(true_classes_CD == 1)}): "
      f"predicted class 0 = {np.sum((true_classes_CD == 1) & (pred_cla
      f"predicted class 1 = {np.sum((true_classes_CD == 1) & (pred_cla

# -----
# Model 2 (C vs D) applied to A/B tiles
# -----
# Same logic in the opposite direction: apply Model 2 to A/B tiles
# and inspect the distribution of its predictions.
preds_CD_on_AB = model_CD.predict(d_gval_AB, verbose=0)
pred_classes_CD_on_AB = np.argmax(preds_CD_on_AB, axis=1)

# Convert generator labels to a numpy array (same reason as above).
true_classes_AB = np.array(d_gval_AB.classes) # 0 = A, 1 = B

# Tabulate predictions broken down by true class. If, for example,
# Model 2 maps almost all A/B tiles to class C (predicted 0), this
# would suggest it has no relevant feature for B-like tissue and is
# defaulting to the C decision in the absence of its blue/D-like cue.
print("\nModel 2 (C vs D) applied to A/B tiles:")
print(f" True A tiles ({np.sum(true_classes_AB == 0)}): "
      f"predicted class 0 = {np.sum((true_classes_AB == 0) & (pred_cla
      f"predicted class 1 = {np.sum((true_classes_AB == 0) & (pred_cla
print(f" True B tiles ({np.sum(true_classes_AB == 1)}): "
      f"predicted class 0 = {np.sum((true_classes_AB == 1) & (pred_cla
      f"predicted class 1 = {np.sum((true_classes_AB == 1) & (pred_cla

```

=====

Prediction distribution (cross-dataset)

=====

Model 1 (A vs B) applied to C/D tiles:

True C tiles (21): predicted class 0 = 17, predicted class 1 = 4

True D tiles (16): predicted class 0 = 16, predicted class 1 = 0

Model 2 (C vs D) applied to A/B tiles:

True A tiles (20): predicted class 0 = 0, predicted class 1 = 20

True B tiles (22): predicted class 0 = 0, predicted class 1 = 22

Q4. Test Model 1 on test set 2 and Model 2 on test set 1. Report the results.

Answer: When each trained model was applied to the other dataset's validation set, performance collapsed dramatically compared with within-domain performance.

Model	Tested on	Accuracy	Loss
Model 1 (A vs B)	Test set 2 (C vs D tiles)	0.4595	57.4434
Model 2 (C vs D)	Test set 1 (A vs B tiles)	0.5238	8.9207

Both models dropped to approximately chance-level accuracy, in stark contrast to their within-domain validation accuracies of 97.62% and 100% respectively. The accompanying cross-entropy losses are also strikingly high, particularly Model 1's loss of 57.44, indicating that the models are not merely guessing at random, but are *confidently incorrect*, assigning high probability to the wrong class on out-of-distribution data.

Examining the prediction distributions provides further insight into *how* each model fails:

Model 1 (A vs B) applied to C/D tiles

True class	Predicted as A-like (0)	Predicted as B-like (1)
C (n = 21)	17	4
D (n = 16)	16	0

Model 1 classified 33 of 37 foreign tiles (89%) as "A-like." Notably, all 16 D tiles were assigned to class A, with none classified as B-like. The observed accuracy of 0.4595 is therefore not the result of meaningful discrimination but largely reflects the numerical alignment between class indices (A and C both mapped to 0).

Model 2 (C vs D) applied to A/B tiles

True class	Predicted as C-like (0)	Predicted as D-like (1)
A (n = 20)	0	20
B (n = 22)	0	22

Model 2 classified every *single* A/B tile (42/42) as "D-like," with no variation in its predictions. This is the most striking result of the experiment, indicating complete collapse to a single class when presented with out-of-distribution data.

Q5. Possible explanations for the observations made in Q4

Answer : The cross-tested models perform poorly because the failure occurs at two distinct levels — a fundamental mismatch in label semantics, and a feature-space drift between domains, compounded by the models' reliance on dataset-specific shortcuts, the limited training data, and the absence of any domain-adaptation strategy, as elaborated below:

1. Label-space mismatch. Before any discussion of domain shift, it is worth noting that the cross-test labels themselves are not commensurable. Model 1's output space is {A, B}, not {C, D}, so when it is applied to C/D patches it has no option to predict the correct class, it must assign each foreign patch to whichever of A or B it most resembles in low-level feature space. Any accuracy reported on this cross-test therefore reflects coincidental colour/texture similarity rather than a biologically grounded decision. The same applies in reverse for Model 2. This issue can be addressed by adopting a shared label space across the two tasks, or by evaluating models on domain-agnostic feature embeddings (e.g. cosine similarity between penultimate-layer representations) rather than direct class predictions.

2. Domain shift in the learned representation. Even setting label semantics aside, the CNN learns texture, colour, and structural features specific to its training data distribution. When tested on a different tissue group, the input distribution shifts and the learned features no longer align. Stacke et al. (2021) demonstrated that the latent representations learned by CNNs on H&E images are highly sensitive to changes in data distribution, and proposed a *representation-shift* metric to quantify this drift across domains. This effect is amplified at small patch sizes, at 100 × 100 px, the network's receptive field cannot capture much high-level morphology, so it relies disproportionately on local stain and texture statistics, which are exactly the features most disrupted by a domain change.

3. Shortcut learning. The prediction distributions reported in Q4 provide direct empirical evidence of *shortcut learning*, a phenomenon formalised by Geirhos et al. (2020) as the tendency of deep networks to learn decision rules that perform well within the training distribution but fail to transfer to more challenging testing conditions. Model 2's classification of all 42 A/B tiles as class D, with zero variation, and Model 1's assignment of 33 of 37 C/D tiles to class A are textbook examples of this behaviour: rather than learning generalisable morphological features, each network appears to have learned a simple low-level rule (for Model 2, plausibly something close to "if not glandular-pink $\rightarrow$ D") that collapses entirely when the relevant cue is absent. The very high cross-entropy losses on cross-test data (57.44 and 8.92) confirm that the models are not merely guessing, they are *confidently incorrect*, which is the diagnostic signature of shortcut learning rather than random failure.

4. Tissue-specific morphology and the limits of cross-tissue transfer.

Without clinical metadata, definitive tissue identification is not possible, but morphological inspection suggests that groups A/B and C/D represent qualitatively different tissue types. C images show features consistent with colonic or intestinal mucosa, with visible crypts, goblet cells, and lamina propria, while D images are dominated by densely packed small lymphocytes consistent with lymphoid tissue (lymph node, tonsil, or a lymphoproliferative lesion). The A/B images are heterogeneous, with at least some tiles showing lymphoid follicles (A1) and adipose tissue alongside glandular structures suggestive of breast (B1). Critically, none of the dominant architectural patterns in C or D (glandular epithelium with goblet cells, or sheets of small lymphocytes) appear in the A/B set in a comparable form. Noorbakhsh et al. (2020) systematically cross-classified CNNs trained on one TCGA cancer type against 18 others and found that transfer success scaled with shared morphology, classifier relationships recapitulated known adenocarcinoma, carcinoma, and developmental groupings, indicating that CNNs latch onto tissue-architecture features that travel across organs only when the underlying biology is shared. The morphological distance between A/B and C/D in this study is therefore exactly the regime in which Noorbakhsh's framework predicts transfer to fail.

5. Stain and colour variation. Even when tissues are biologically related, differences in staining protocols, reagent concentrations, and digitisation equipment introduce colour shifts that confuse CNNs. Tellez et al. (2019) showed that training without stain colour augmentation or normalisation leads to poor generalisation across laboratories, and that stain-specific augmentation techniques can substantially improve cross-domain performance.

6. Small dataset size. With only two source images per class, the CNNs are

exposed to a very limited range of tissue and staining variation and are prone to overfitting dataset-specific artefacts (e.g., background intensity, tile edge effects, batch-specific stain hue) rather than biologically meaningful features that might generalise.

7. No domain adaptation. The models were trained in a standard supervised setting with no techniques to encourage domain-invariant feature learning. Methods such as stain normalisation, stain colour augmentation, domain-adversarial training, or transfer learning from large pretrained pathology foundation models could potentially improve cross-dataset performance.

References

1. Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M. and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665–673. <https://doi.org/10.1038/s42256-020-00257-z>
2. Noorbakhsh, J., Farahmand, S., Foroughi pour, A. et al. (2020). Deep learning-based cross-classifications reveal conserved spatial behaviors within tumor histological images. *Nature Communications*, 11, 6367. <https://doi.org/10.1038/s41467-020-20030-5>
3. Stacke, K., Eilertsen, G., Unger, J. and Lundström, C. (2021). Measuring Domain Shift for Deep Learning in Histopathology. *IEEE Journal of Biomedical and Health Informatics*, 25(2), 325–336. <https://doi.org/10.1109/JBHI.2020.3032060>
4. Tellez, D., Litjens, G., Bándi, P., Bulten, W., Bokhorst, J., Ciompi, F. and van der Laak, J. (2019). Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology. *Medical Image Analysis*, 58, 101544. <https://doi.org/10.1016/j.media.2019.101544>

The End

In []: